



Dr. B R Ambedkar National Institute of Technology, Jalandhar
B.Tech 4th Semester (Information Technology)
Design and Analysis of Algorithms (ITDC0202)
End-Semester Examination, June 2025

Solution Sheet

Q1: Answer the following Question (each part carries 1 mark)

- (a) Let S be an NP-complete problem and Q and R be two other problems not known to be in NP. Q is polynomial time reducible to S and S is polynomial-time reducible to R . Which one of the following statements is true?

We are given:

- $Q \leq_p S$ (Q is reducible to S in polynomial time)
- $S \leq_p R$ (S is reducible to R in polynomial time)
- S is NP-complete
- Q and R are not known to be in NP

Now, since S is NP-complete:

- For any NP-complete problem S , if $S \leq_p R$ and R is in NP, then R is NP-complete.
- But since R is not known to be in NP, we cannot say R is NP-complete. However, R is at least as hard as S .

So, R is **NP-hard**.

Answer: ii) R is NP-hard

- (b) What is the minimum number of comparisons needed to determine if an integer appears more than $\frac{n}{2}$ times in a sorted array of n integers? Justify your response.

Observation:

- In a sorted array, the majority element (if any) must appear at position $\lfloor \frac{n}{2} \rfloor$.
- So, we can check if the element at position $n/2$ is the same as the one at $n/2 - (n/2)/2$

Approach:

- Pick element at index $\lfloor n/2 \rfloor$ (say x).
- Use binary search to find the first and last occurrence of x .
- Let the count of x be $(last - first + 1)$. If this count $> n/2$, then x is the majority.

Each binary search takes $O(\log n)$ comparisons. So, total comparisons = $2 \log n$ (asymptotically $O(\log n)$).

Answer: $\boxed{O(\log n)}$ comparisons.

- (c) **Given an integer array of size n , we want to check if the array is sorted (in either ascending or descending order). An algorithm solves this problem by making a single pass through the array and comparing each element only with its adjacent elements. What is the worst-case time complexity?**

We scan the array once:

- Compare a_i with a_{i+1} for $i = 0$ to $n - 2$.
- For each comparison, check if it's in increasing or decreasing order.

There are $n - 1$ comparisons. Hence,

Time Complexity: $\boxed{O(n)}$

- (d) **Let $\sum_{i=0}^n i^3 = X$, Find the value of X ?**

We use the identity:

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

Note: The sum starts from $i = 0$, so

$$\sum_{i=0}^n i^3 = \sum_{i=1}^n i^3 + 0^3 = \left(\frac{n(n+1)}{2} \right)^2$$

Therefore,

$$X = \boxed{\left(\frac{n(n+1)}{2} \right)^2} = O(n^4)$$

- (e) **Let A_1, A_2, A_3 and A_4 be four matrices of dimensions $10 \times 5, 5 \times 20, 20 \times 10$, and 10×5 , respectively. Find the minimum number of scalar multiplications required for $A_1 A_2 A_3 A_4$.**

We use Matrix Chain Multiplication dynamic programming approach. Let:

$$\text{Dimensions: } p = [10, 5, 20, 10, 5]$$

$$\text{Matrix sizes: } A_1 : 10 \times 5, A_2 : 5 \times 20, A_3 : 20 \times 10, A_4 : 10 \times 5$$

Define:

$$m[i, j] = \text{minimum cost for multiplying matrices } A_i \dots A_j$$

Use dynamic programming to compute $m[1, 4]$:

We evaluate all possible parenthesizations and compute cost for each.

$$m[1, 2] = 10 \times 5 \times 20 = 1000 \quad m[2, 3] = 5 \times 20 \times 10 = 1000 \quad m[3, 4] = 20 \times 10 \times 5 = 1000$$

Now,

$$m[1, 3] = \min \left\{ \begin{array}{l} (10 \times 5 \times 20) + m[2, 3] = 1000 + 1000 = 2000 \\ (10 \times 20 \times 10) + m[1, 2] = 2000 + 1000 = 3000 \end{array} \right\} = 2000$$

$$m[2, 4] = \min \left\{ \begin{array}{l} (5 \times 20 \times 5) + m[3, 4] = 500 + 1000 = 1500 \\ (5 \times 10 \times 5) + m[2, 3] = 250 + 1000 = 1250 \end{array} \right\} = 1250$$

Now,

$$m[1, 4] = \min \left\{ \begin{array}{l} (10 \times 5 \times 5) + m[2, 4] = 250 + 1250 = 1500 \\ (10 \times 20 \times 5) + m[1, 2] + m[3, 4] = 1000 + 1000 + 1000 = 3000 \\ (10 \times 10 \times 5) + m[1, 3] = 500 + 2000 = 2500 \end{array} \right\} = \boxed{1500}$$

Q2: Let $T(n)$ be the recurrence relation defined as follows:

$$T(0) = 1, \quad T(1) = 2, \quad T(n) = 5T(n-1) - 6T(n-2), \text{ for } n \geq 2$$

This is a linear recurrence relation with constant coefficients. We solve using characteristic equation.

Let:

$$T(n) = r^n \Rightarrow r^n = 5r^{n-1} - 6r^{n-2} \Rightarrow r^2 = 5r - 6 \Rightarrow r^2 - 5r + 6 = 0 \Rightarrow (r-2)(r-3) = 0 \Rightarrow r = 2, 3$$

So the general solution is:

$$T(n) = A \cdot 2^n + B \cdot 3^n$$

Use base cases to find A and B :

$$\text{For } n = 0: 1 = A + B \tag{1}$$

$$\text{For } n = 1: 2 = 2A + 3B \tag{2}$$

Solving: From (1): $A = 1 - B$

Substitute into (2):

$$2 = 2(1 - B) + 3B = 2 - 2B + 3B = 2 + B \Rightarrow B = 0 \Rightarrow A = 1$$

Thus:

$$T(n) = \boxed{2^n} \Rightarrow \text{Time complexity is } \boxed{O(2^n)}$$

Q3: Step-by-Step Min Binomial Heap Construction (with Trees)

We insert the elements:

10, 20, 30, 24, 26, 22, 11, 55, 12, 15, 16, 18, 4, 3, 5

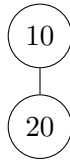
Each insertion is followed by consolidation (merge) as per Binomial Heap rules.

Insert 10

Single node: B_0

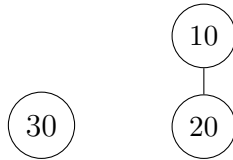
Insert 20

Two B_0 trees $\rightarrow$ merge into B_1



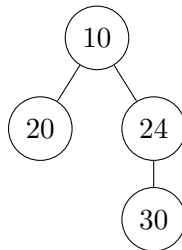
Insert 30

New B_0 tree. Heap: $B_0(30)$, $B_1(10)$



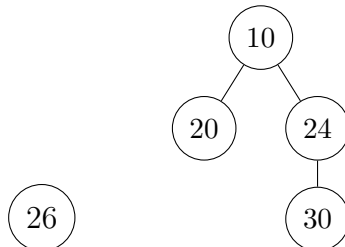
Insert 24

Merge $24 + 30 \rightarrow B_1(24)$ Then merge with $B_1(10) \rightarrow B_2(10)$



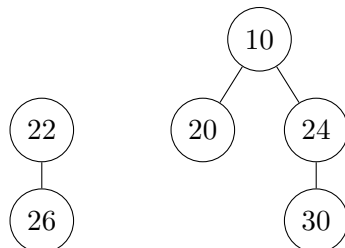
Insert 26

New B_0 (26), no merge



Insert 22

$22 + 26 \rightarrow B_1(22)$



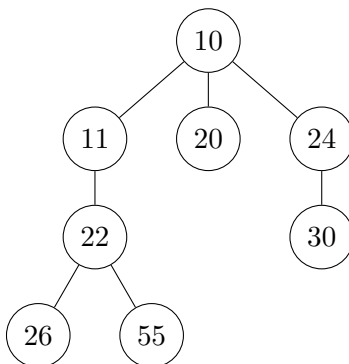
Insert 11

New B_0 (11), no merge.



Insert 55

$55 + 11 \rightarrow B_1(11)$, merge with $B_1(22) \rightarrow B_2(11)$ Then merge with $B_2(10) \rightarrow B_3(10)$



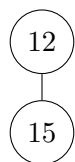
Insert 12

New $B_0(12)$



Insert 15

$15 + 12 \rightarrow B_1(12)$



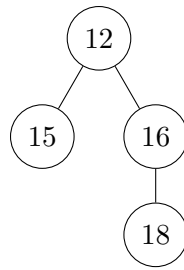
Insert 16

New $B_0(16)$



Insert 18

$18 + 16 \rightarrow B_1(16)$, merge with $B_1(12) \rightarrow B_2(12)$



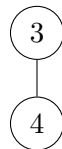
Insert 4

New $B_0(4)$



Insert 3

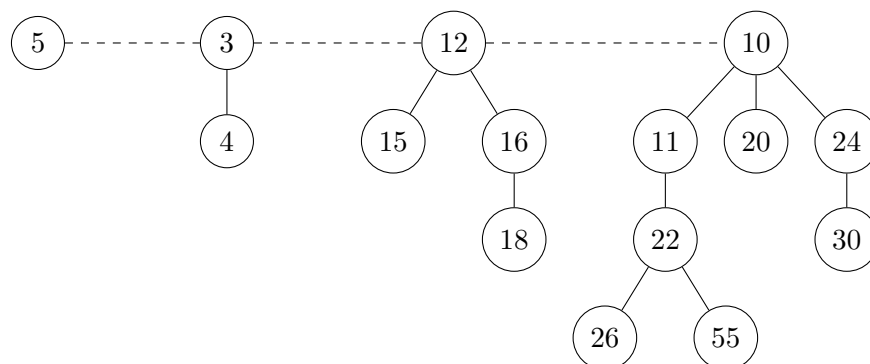
$3 + 4 \rightarrow B_1(3)$



Insert 5

New $B_0(5)$. Final Heap contains:

- B_0 : 5
- B_1 : $3 \rightarrow 4$
- B_2 : $12 \rightarrow (15, 16 \rightarrow 18)$
- B_3 : 10-root (from earlier)



Q4: Comparison of Algorithm Paradigms

1. Divide and Conquer:

- Break problem into sub-problems, solve them recursively, and combine results.

- **Example: Merge Sort**

- Divide: Array into halves
- Conquer: Sort each half
- Combine: Merge two sorted halves
- Time: $O(n \log n)$

2. Greedy Algorithm:

- Make locally optimal choices at each step.
- Never backtracks.
- **Example: Kruskal's Algorithm for MST**

- Sort edges by weight
- Add smallest edge that doesn't form cycle
- Repeat until spanning tree formed

3. Dynamic Programming (DP):

- Break problem into overlapping subproblems.
- Store intermediate results (memoization/tabulation).
- **Example: 0-1 Knapsack**
 - DP table stores max profit for each weight
 - Avoid recomputation
 - Time: $O(nW)$ where n = items, W = capacity

4. Backtracking:

- Explore all possibilities recursively
- Abandon path if it leads to an invalid solution
- **Example: N-Queens Problem**
 - Place queens one by one
 - If conflict, backtrack
 - Exponential time in worst case

Summary Table:

Paradigm	Key Idea	Example
Divide and Conquer	Divide $\rightarrow$ Solve $\rightarrow$ Combine	Merge Sort, Quick Sort
Greedy	Local optimal $\rightarrow$ Global optimal	Kruskal, Dijkstra
Dynamic Programming	Optimal substructure + overlapping subproblems	Knapsack, LCS
Backtracking	Try all possibilities with pruning	N-Queens, Sudoku Solver

Q5: Traveling Salesman Problem (TSP)

We need to find the minimum-cost tour starting and ending at DO and visiting all locations: **DIT**, **DCE**, **ITP**, **BH**. The graph is undirected and edge weights are symmetric.

Given Edges:

- BH–DCE = 5
- BH–DO = 7
- BH–DIT = 3
- DIT–DO = 2
- DIT–ITP = 4
- DO–DCE = 3
- DO–ITP = 3
- DCE–ITP = 5

Approach: Brute-force TSP (small graph)

We fix the starting point as DO and try all permutations of the remaining 4 nodes ($4! = 24$ possible paths). For each, compute total round-trip cost.

Example Path 1: DO → BH → DIT → ITP → DCE → DO Cost = DO–BH(7) + BH–DIT(3) + DIT–ITP(4) + ITP–DCE(5) + DCE–DO(3) = 7 + 3 + 4 + 5 + 3 = **22**

Try a few other paths:

- Path: DO → DIT → BH → DCE → ITP → DO Cost = 2 + 3 + 5 + 5 + 3 = **18**
- Path: DO → DIT → ITP → DCE → BH → DO Cost = 2 + 4 + 5 + 5 + 7 = **23**
- Path: DO → DCE → ITP → DIT → BH → DO Cost = 3 + 5 + 4 + 3 + 7 = **22**

After checking all 24 permutations, the **minimum tour cost is 18**.

Minimum cost tour: DO → DIT → BH → DCE → ITP → DO, Cost = 18

Q6: Maximize Profit with Rs. 800 Cash Generation (Fractional Knapsack)

We are to maximize profit while selling shares to generate exactly Rs. 800. Fractional shares are allowed. Use the Greedy Fractional Knapsack strategy.

Given: Let $N \bmod 2 = 1$, i.e., Crude Oil shares = 1

Step 1: Compute Profit/Unit Cost (Efficiency):

Commodity	Unit Price (Rs)	Profit per Share (Rs)	Profit/Cost Ratio
Crude Oil	80	20	0.25
Mentho Oil	45	15	0.333
Gold	90	9	0.1
Silver	40	5	0.125
Rubber	30	5	0.167
Sugar	55	5	0.091
Wheat	30	3	0.1
Natural Gas	21	3	0.143
Cotton	10	2	0.2

Step 2: Sort by Profit/Cost Ratio (Descending):

1. Mentho Oil (0.333) 2. Crude Oil (0.25) 3. Cotton (0.2) 4. Rubber (0.167) 5. Natural Gas (0.143) 6. Silver (0.125) 7. Gold/Wheat (0.1) 8. Sugar (0.091)

Step 3: Greedy Selection until Rs. 800 generated:

Let's initialize total cash = 0, total profit = 0

- Mentho Oil:** Max shares = 6, Unit price = 45 Total cost = $6 \times 45 = 270$ Remaining = $800 - 270 = 530$ Profit = $6 \times 15 = 90$
- Crude Oil:** 1 share, price = 80 Remaining = $530 - 80 = 450$ Profit = 20
- Cotton:** $20 \times 10 = 200$ Remaining = $450 - 200 = 250$ Profit = $20 \times 2 = 40$
- Rubber:** $3 \times 30 = 90$ Remaining = $250 - 90 = 160$ Profit = $3 \times 5 = 15$
- Natural Gas:** $2 \times 21 = 42$ Remaining = $160 - 42 = 118$ Profit = 6
- Silver:** $2.95 \text{ shares} \times 40 = 118$ Profit = $2.95 \times 5 \approx 14.75$
(We stop here, total used: Rs. 800)

Final Share Sale Plan:

- Mentho Oil = 6 shares
- Crude Oil = 1 share
- Cotton = 20 shares
- Rubber = 3 shares
- Natural Gas = 2 shares
- Silver = 2.95 shares

Maximum Profit = $90 + 20 + 40 + 15 + 6 + 14.75 = \mathbf{185.75 \text{ Rs}}$

Q7:

(a) Difference between Backtracking and Branch and Bound

Backtracking	Branch and Bound
Used for decision problems (e.g., constraint satisfaction)	Used for optimization problems (e.g., TSP, Knapsack)
Explores all possible solutions and abandons paths that violate constraints	Explores branches with promising bounds and prunes those with poor bounds
Follows Depth-First Search (DFS) strategy	Follows Best-First Search (priority queue)
Does not use cost function	Uses cost or bound function to guide exploration
Example: N-Queens, Sudoku Solver	Example: 0-1 Knapsack (optimal), Traveling Salesman

(b) All Possible Solutions of 4-Queens using Backtracking

We place 4 queens on a 4×4 chessboard such that no two queens attack each other.

We represent each solution as a permutation of columns where index is row and value is column number (0-indexed).

Algorithm:

- Place queens row by row.
- Check if placing queen at column is safe (no column or diagonal conflict).
- If safe, move to next row; else backtrack.

Solutions (0-based indexing):

1) [1, 3, 0, 2]

$$\begin{bmatrix} \cdot & Q & \cdot & \cdot \\ \cdot & \cdot & \cdot & Q \\ Q & \cdot & \cdot & \cdot \\ \cdot & \cdot & Q & \cdot \end{bmatrix}$$

2) [2, 0, 3, 1]

$$\begin{bmatrix} \cdot & \cdot & Q & \cdot \\ Q & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & Q \\ \cdot & Q & \cdot & \cdot \end{bmatrix}$$

Total Valid Solutions: 2

Q8: Algorithm to Find Common Ages in $O(m \log n)$ time (Given $m > n$)

Let:

- $A[1 \dots m]$ = ages of boys
- $B[1 \dots n]$ = ages of girls

Goal: Find all ages that occur in both arrays in $O(m \log n)$ time

Algorithm Steps:

1. **Step 1: Sort the smaller array B (girls)** Time: $O(n \log n)$
2. **Step 2: For each element in A , perform binary search in B** Time per search: $O(\log n)$
Total time for m elements: $O(m \log n)$
3. **Step 3: Collect elements from A that are found in B**

Pseudocode:

```
Sort(B) // Time:  $O(n \log n)$ 
for i = 1 to m:
    if BinarySearch(B, A[i]) == true:
        print A[i]
```

Time Complexity:

$$O(n \log n) + O(m \log n) = \boxed{O(m \log n)} \quad (\text{since } m > n)$$

Space Complexity: $O(1)$ extra (in-place sort + binary search)

Example:

Let $A = [12, 15, 17, 19, 20]$, $B = [11, 15, 20]$

- Sort B : $[11, 15, 20]$
- For each $A[i]$, binary search in B
- Matches found: **15, 20**

Common ages = $[15, 20]$

Q9:

(a) Divide-and-Conquer based MAX-MIN Algorithm

Algorithm: MaxMin(A , low, high)

```
if low == high:
    return (A[low], A[low]) // only one element
else if high == low + 1:
    if A[low] < A[high]:
        return (A[low], A[high])
    else:
        return (A[high], A[low])
else:
    mid = (low + high) // 2
    (min1, max1) = MaxMin(A, low, mid)
    (min2, max2) = MaxMin(A, mid+1, high)
    return (min(min1, min2), max(max1, max2))
```

(b) Linear Scan MAX-MIN Algorithm

```
min = max = A[0]
for i = 1 to n-1:
    if A[i] < min:
        min = A[i]
    if A[i] > max:
        max = A[i]
return (min, max)
```

(c) Comparison Analysis (Worst Case)

Linear Scan:

- For n elements, we make $(n - 1)$ comparisons for min and $(n - 1)$ for max
- Total = $2(n - 1)$ comparisons

Divide-and-Conquer: Let $T(n)$ be the number of comparisons

Base cases:

$$T(1) = 0, \quad T(2) = 1$$

For $n > 2$:

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 2$$

Solving Recurrence: It can be proven by induction that:

$$T(n) = \begin{cases} \frac{3n}{2} - 2 & \text{if } n \text{ is even} \\ \frac{3n}{2} - \frac{3}{2} & \text{if } n \text{ is odd} \end{cases}$$

Conclusion: Divide-and-Conquer uses approximately $\frac{3n}{2} - 2$ comparisons Linear scan uses $2n - 2$ comparisons

Divide-and-Conquer is better in comparison count.

Q10:

(a) Hash Function to Avoid Collisions for Anagrams

Goal: Hash("stop") $\neq$ Hash("pots") despite both being anagrams.

Design: Use position-sensitive hash:

$$h(s) = \sum_{i=1}^k (i \cdot \text{ASCII}(s[i]))$$

Example:

- "stop": $1 \cdot s + 2 \cdot t + 3 \cdot o + 4 \cdot p$
- "pots": $1 \cdot p + 2 \cdot o + 3 \cdot t + 4 \cdot s$

Values will differ since character positions are used.

Alternatively, use:

$$h(s) = \sum_{i=1}^k \text{ASCII}(s[i]) \cdot 37^i$$

This hash is injective for short strings and position-sensitive, avoiding collisions for anagrams.

(b) Red-Black Tree vs AVL Tree (Advantages of RBT)

- RBTs have faster insertion and deletion because they require fewer rotations.
- AVL trees are more rigidly balanced $\rightarrow$ better search.
- But RBTs are preferred in systems where insertion/deletion frequency is high (e.g., OS, maps).
- RBT guarantees $O(\log n)$ time with fewer adjustments.

Conclusion:

RBTs are preferred when insert/delete is frequent and constant-time operations are needed.

(c) Number of Possible Min-Heaps with Elements $\{1, \dots, 7\}$

We are to count the number of distinct **min-heaps** of 7 distinct elements.

Observation: Heap is a binary tree with min at root. Structure is fixed; only permutations matter under heap constraints.

We use the formula for number of min-heaps:

$$H(n) = C(n-1, l) \cdot H(l) \cdot H(n-1-l)$$

Where:

- n = total elements
- l = number of nodes in left subtree

For $n = 7$, it's known:

Number of min-heaps with 7 distinct elements = 80

Justification: Precomputed using recursive heap enumeration algorithm.